# CS2109S Cheatsheet

AY24/25 - Anselm Long

# Searching

## Uninformed Search

- **Breadth First Search** - Frontier is a queue. Complete if b is finite, optimal if cost is 1 per step. TC and SC = $O(b^{d+1})$. Might not terminate if no solution.
- **Uniform Cost Search** - Frontier is a priority queue. Complete if positive step cost everywhere and finite total cost, optimal if positive step cost everywhere. TC and SC exponential wrt tier of optimal solution. Might not terminate.
- **Depth First Search** - Frontier is a stack. Not complete, depth of search tree may be infinite. Not optimal, solution might be at a shallower depth. TC exponential, SC polynomial wrt max depth.
- **Depth Limited Search** - Frontier depends. Not complete, not optimal if DFS is used. TC exponential, SC polynomial if DFS used.
- **Iterative Deepening Search** - Frontier depends. Complete, optimal if step cost is equal throughout. TC polynomial, SC polynomial if DFS. Might not terminate.

## Informed Search

Guides search through a heuristic - estimated of the optimal path cost from any state to goal state.

**Properties of a Heuristic**

- **Admissibility** - never overestimates the cost to reach a goal. An admissible heuristic for a relaxed problem must also be admissible for a more constrained problem. If $h(n)$ is admissible, then A* without visited memory is optimal.
- **Consistency** - for every node $n$ and successor $n'$, $h(n) \leq c(n, a, n') + h(n')$, $h(n) - h(n') \leq c(n, a, n')$. If a heuristic is consistent, it is admissible. Not admissible $\implies$ not consistent. If $h(n)$ is consistent, A* with visited memory is optimal.
- **Dominance** - $h_1$ dominates $h_2$ if $h_1(n) \geq h_2(n)$

**Informed Search Algorithms**

- **Best First Search** - Frontier is a priority queue with evaluation function $f(n)$
- **A* Search** - Best First Search with evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is path cost from initial node to node $n$, and $h(n)$ is estimated cost from $n$ to the goal. Complete if edge costs are positive and branching factor is finite. Optimality depends on the heuristic. TC, SC, exponential.

# Local Search

**Definition** - traverse state space by considering locally reachable states based on an evaluation function. Incomplete and suboptimal.

**Any-time property** - longer runtime, better solution. Can obtain a good enough solution for difficult problems.

**Problem Formulation** - States, initial state, goal test, successor function

**Successor Function** - generates neighboring states by applying small modifications or changes from the current state.

**Hill Climbing Algorithm (Greedy Local Search)** - searches neighbors until the highest valued successor of the current state is less than the current state's evaluation, meaning the current state is the best state. Escape techniques: Random Restarts, Random Walk

# Adversarial Search

## Minimax

Assume both players play optimally. Complete and optimal. TC expo, SC poly.

**Alpha-beta Pruning** - prune subtree when it doesn't affect the decision. $\alpha$ = highest value choice so far for MAX. $\beta$ = lowest value choice so far for MIN. At a MAX node, we can stop when we find a node $\geq \beta$. At a MIN node, we stop when we find a node $\leq \alpha$. $O(b^{\frac{m}{2}})$ with perfect ordering.

# Machine Learning

**Types of Problems**

- **Classification** - output one of a finite set of values, discrete
- **Regression** - outputs a number, continuous

**Dataset** - a set of pairs, $(x_i, y_i)$ where $x_i$ is the input vector, $y_i$ is the target.

We assume there is an underlying relationships between $y$ and $x$, $y = f(x) + \epsilon$. We try to find a function that approximates $f(x)$.

**Performance Measure Error Functions:** Mean Square Error $= \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$, Mean Absolute Error $= \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$ MSE is more volatile to outliers.

**Confusion Matrix**

| | Actual + | Actual − |
|---|---|---|
| Pred + | TP | FP |
| Pred − | FN | TN |

**Metrics: Accuracy** - $\frac{TP+TN}{TP+FN+FP+TN}$ (Might not be good if dataset is skewed) **Precision** - $\frac{TP}{TP+FP}$ (Maximises if FP is costly, email spam) **Recall** - $\frac{TP}{TP+FN}$ (Maximise if FN is dangerous eg. cancer prediction) **F1 Score** - $\frac{2}{\frac{1}{P}+\frac{1}{R}}$

# Decision Trees

## Entropy

A measure of randomness. Increased randomness means higher entropy. Let a dataset contain $p$ positive and $n$ negative examples. $P(pos) = \frac{p}{p+n}$, $P(neg) = \frac{n}{p+n}$

$$I(A, B) = -Alog_2 A - Blog_2 B$$

**Information Gain** = Entropy before dividing - average entropy after dividing (remainder)

$$IG(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - \text{remainder}(A)$$

$$\text{remainder}(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p+n} I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$$

**Decision Tree Learning** - recursively choose the attribute with the best information gain. Prone to **overfitting**, remedy with pruning (max depth or min sample leaf). Pre-process data to remove outliers that create noise.

# Linear Regression

## Linear Model

Hypothesis class of linear models, given an input vector x of dimension d is given by

$$h_w(x) = w_0x_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

Using dot product, this becomes $h_w(x) = w^T \cdot x$.

## Feature Transformations

- **Feature Engineering** - create new features based on existing features, eg. polynomial, log, exp
- **Feature Scaling** - scale to be within a specific range

## Measuring Fit

We use the mean squared error loss function

$J_{MSE}(w) = \frac{1}{2m}\sum_{i=1}^{m}(h_w(x^{(i)}) - y^{(i)})^2$,

$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(h_w(x^{(i)}) - y^{(i)})x_j^{(i)}$,

$J_{MSE}(w) = \frac{1}{2m}\sum(w_1x^{(i)} - y^{(i)})^2$,

$\frac{\partial J}{\partial w_1} = \frac{1}{m}\sum(w_1x^{(i)} - y^{(i)})x^{(i)}$, $\nabla J(w) = \begin{bmatrix} \partial J/\partial w_0 \\ \partial J/\partial w_1 \\ \vdots \end{bmatrix}$

To minimize loss, i.e. $f'(x) = 0$, we find a $w$ using these 2 methods below.

## Learning via Normal Equations

Assuming $X^TX$ is invertible: $w = (X^TX)^{-1}X^Ty$

Problems:

- High computational cost - $d^3$
- $X^TX$ non-invertible when it is singular - when rows or column are linearly dependent (highly correlated features)
- Does not work for non-linear models

## Learning via Gradient Descent

Compute an estimate of the gradient at each point and move a small amount in the steepest downhill direction.

$$w_j \leftarrow w_j - \gamma\frac{\partial J(w_0, w_1, \ldots)}{\partial W_j}$$

Set an appropriate learning rate ($\gamma$) that isn't too large (oscillating) or too little (takes long to converge).

$$w_0 \leftarrow w_0 - \gamma\frac{1}{m}\sum_{j=1}^{m}(h_w(x_j) - y_j)$$

$$w_1 \leftarrow w_1 - \gamma\frac{1}{m}\sum_{j=1}^{m}(h_w(x_j) - y_j) \times x_j$$

**Convexity** - if the line segment between any two distinct points on the graph of the function lies above or on the graph between the two points. A convex function has a single global minimum. MSE loss function is **convex** for linear regression and polynomial regression. Feature scaling needed to standardize different scales.

## Variants of Gradient Descent

- **Batch Gradient Descent** - considering all training examples
- **Mini-batch GD** - consider subset of training examples at a time. Cheaper with some randomness
- **Stochastic GD** - select one random data point at a time, more randomness, needs more steps

# Logistic Regression

## Model

Linear functions can be used to do **classification**. A decision boundary is a line or surface that separates two classes. We can have the classification hypothesis:

$$h_w(x) = \sigma(w_0x_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d)$$

where $x_0 = 1$ is a dummy variable, and the shorthand is $h_w(x) = \sigma(w^Tx)$.

However, since MSE is **non-convex** for logistic regression, we use **Binary Cross Entropy, as shown:**

$$BCE(y, \hat{y}) = -y\ log(\hat{y}) - (1-y)\ log\ (1-\hat{y})$$

Then, we define the mean BCE:

$$J_{BCE}(w) = \frac{1}{N}\sum_{i=1}^{N} BCE(y_i, h_w(x_i))$$

Known as BCE Loss, this is **convex** for logistic regression. Gradient of loss function is the same as linear regression's.

## Multi-Class Classification

**One-vs-One**

A separate binary classifier is trained for **every pair of classes**. For $C$ classes, results in $\frac{C(C-1)}{2}$ classifiers. Class with the most votes is selected.

**One-vs-Rest**

We can train $C$ different logistic classifiers for all $C$ classes, then for each input, pick the class $i$ with the maximum output from their hypothesis.

**Advanced Topics**

- **Generalization** - model's performance on unseen data
  - Dataset Quality and Quantity - relevance, noise, balance
  - Model Complexity - Bias and Variance
    - Bias – some difference or error occurring between the model's predicted value and the actual value. High bias: model is too simple to capture complex ground truth.
    - Variance – measure of spread in data from its mean position. High variance: retraining a complex model with a different training set can lead to a different model.
- Hyperparameters - settings that control the behavior of the training algorithm but are not learned from the data, need to be set before training begins.
  - Tuning - optimizing the hyperparameters to improve its performance

# Support Vector Machines

## Regularization

$$J_{reg}(w) = J(w) + \lambda P(w)$$

Ridge Regression heavily penalizes larger parameters, to pull all parameters towards smaller values while Lasso Regression zeros out certain parameters, implicitly selecting features to be 'excluded' and behaves as a feature selection strategy. This can be applied to Logistic Regression too.

**Penalty Function,** $P(w)$

- **Ridge Regression** - $\sum\frac{1}{2}w_j^2$
- **Lasso Regression** - $\sum|w_j|$

## Normal Equation with Regularization

$$w = (X^TX + \lambda I)^{-1}X^TY$$

For all $\lambda > 0$ the matrix $(X^TX + \lambda I)^{-1}$ is invertible.

## Kernels

Normal equation can be written as a weighted (linear) combination of training data.

**Polynomial Kernel** - $K(u, v) = (u \cdot v)^d = \phi(u) \cdot \phi(v)$

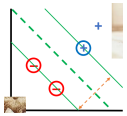
**Gaussian Kernel** - $K(u, v) = e^{-\frac{||u-v||^2}{2\sigma^2}} = \phi(u) \cdot \phi(v)$

**Transformed Features** - if we take a d-dimensional feature vector and transform it into a M-dimensional feature vector where $M \geq d$, $x \in \mathbb{R}^d \rightarrow \phi(x) \in \mathbb{R}^M$.

**Kernel Trick** - when a dot product shows up, we can replace it with other similarity functions $k(u, v)$. A valid kernel = *continuous symmetric positive-definite kernel.* Kernel trick allows us to separate data in the SVM hyperplane that is otherwise not linearly separable. We don't have to compute $\phi(x)$ explicitly. For any valid kernel, there exists a corresponding feature transformation where $k(u, v) = \phi^T(u)\phi(v)$

## Hyperplanes

SVMs aim to maximize the margin between positive and negative samples.



**Prediction Rule:** $f(x) = \text{sign}(w^Tx + b)$

**Margin:** $\frac{|w^Tx+b|}{\|w\|}$, support vectors satisfy $y^{(i)}(w^Tx^{(i)} + b) = 1$

**Constraint:** $y^{(i)}(w^Tx^{(i)} + b) \geq 1$ for all $i$

**Objective:** minimize $\frac{1}{2}\|w\|^2$ subject to the constraint above

Defined by a normal vector $w$ and contains all vectors $u$ for which $w^Tu = 0$. Distance to hyperplane: $\frac{|w^Tu|}{||w||}$

Points on the edge of hyperplane - **support vectors**. The rest are not used.

# Unsupervised Learning

## K-means Clustering

**Centroid** - average of a set of points in a cluster.

**Algorithm:**

1. Randomly initialize $K$ centroids (chosen using elbow method)
2. Repeat until convergence
   (a) Assign points to the nearest centroids (have deterministic tiebreaker)
   (b) Recompute - move centroids to average of distances from data points

**Distortion** - average square distance of each sample to its assigned centroid: $\frac{1}{N}\sum||x_i - \mu_{c_i}||^2$. This function can be changed to another distance based function. Each step in the K-Means algorithm never increases distortion.

**Local Optima Problem** - run multiple times and choose clusters with minimum loss. Not guaranteed to reach global minimum. **Variants:** Pick initial centroids from points in the data/**K-medoids**, new centroid is data point closest to current centroid.

## Dimensionality Reduction

**Curse of Dimensionality:** number of samples increase exponentially with number of features

## Singular Value Decomposition

For any $d \times N$ real valued matrix A, there exists a factorization

$$X^T = A = U\Sigma V^T$$

such that $U$ is $d \times d$ and has *orthonormal* columns, $\Sigma$ is $d \times N$ and is a diagonal matrix with $\sigma_j \geq 0$, and V is $N \times N$ and has $N$ *orthonormal* columns and rows. We remove less important basis vectors to create a truncated matrix $\tilde{\Sigma}$ that implies a smaller basis matrix, $\tilde{U}$. We use this to compress the data matrix $X^T$ by $Z := \tilde{U}^T X^T$, where $Z$ is $r \times N$ matrix.

- Truncated SVD: $X \approx \tilde{U}\tilde{\Sigma}V^T$
- Reduced data: $Z = \tilde{U}^T X$
- Reconstruct: $\tilde{X} = \tilde{U}Z = \tilde{U}\tilde{U}^T X$

**Reconstruction** - for a fixed $r$, it holds that $\tilde{U}\tilde{U}^T \approx \mathbb{I}$, depending on $r$. **Mean–centered Data** - compute mean feature vector, compute mean centered data points, then use mean-centered data matrix. Given a mean centered data matrix, the values $\frac{\sigma_j^2}{N-1}$ are variances of the data in the basis defined by the vectors $u_j$.

To retain at least 99% of variance in the data, we choose a minimum $r$ such that $\frac{\sum_{i=1}^{r}\sigma_i^2}{\sum_{i=1}^{N}\sigma_i^2} \geq 0.99$. Using this $r$, original data points (mean centered) and reconstructed data points are close as follows:

$$\frac{\sum_{i=1}^{N} ||\hat{x}_i - \tilde{x}_i||^2}{\sum_{i=1}^{N} ||\hat{x}_i||^2} \leq 0.01$$

**Principal Component Analysis** - statistical application of SVD

# Neural Networks

## Perceptron Learning Algorithm

1. Initialize $w$
2. Loop until convergence or max steps reached
   (a) For each instance of $(x_i, y_i)$, classify $\hat{y}_i = h_w(x_i)$.
   (b) Select one **misclassified** instance $(x_k, y_k)$.
   (c) Update weights: $w \leftarrow w + \gamma(y_k - \hat{y}_k)x_k$.

Remember to add 1 for bias term. Only uses sign function as activation function. If data is not linearly separable, algorithm will not converge.

Activation Functions: Sigmoid (vanishing gradient problem), tanh, ReLU (leads to piecewise linear boundaries), Leaky ReLU... Single-layer NN can be used for a linearly separable model, or even classification. Derivative of sigmoid $\sigma'(x) = \sigma(x)(1 - \sigma(x))$. Multi-layer NN used for more complex models that are not linearly separable.

## Multi-Class Classification

$$g(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{c} e^{z_j}} \in [0, 1]$$

Softmax - converts raw scores into probabilities where the output values sum to 1.

## Neural Networks Training

**Gradient Computation**

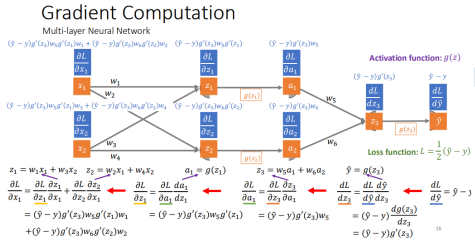
1. Generate predicted value for a given data point (x, y)

$$\hat{y} = g(\sum_{j=0}^{d} w_j x_j) \rightarrow \hat{y} = g(z)$$

2. Define the loss function, eg. MSE
3. Then, gradient of loss function wrt $w_j$ is

$$\frac{\partial L}{\partial w_j} = \frac{dL}{d\hat{y}}\frac{d\hat{y}}{dz}\frac{\partial z}{\partial w_j} = (\hat{y} - y)g'(z)x_j$$

**Forward Propagation** - propagate result from previous layer to next layer until output layer reached.
**Backpropagation** - Compute the gradients of the loss function with respect to each weight.




FP to compute values through multiplication, convert non-linear activation functions $g(x)$ to $g'(x)$, reverse graphs and compute gradients.

**PyTorch**

```
class NeuralNetRegressor(torch.nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.linear1 = torch.nn.Linear(input_size, hidden_size, bias=False)
        self.linear2 = torch.nn.Linear(hidden_size, 1, bias=False)
        self.relu = torch.nn.ReLU()

    def forward(self, x):
        f1 = self.linear1(x)
        a1 = self.relu(f1)
        f2 = self.linear2(a1)
        return f2

model1 = NeuralNetRegressor(2,8) # 2 features, 8 hidden neurons
```

x ∈ ℝ² → Linear → ReLU → Linear → y ∈ ℝ

same

```
model2 = torch.nn.Sequential(
            torch.nn.Linear(2,8),
            torch.nn.ReLU(),
            torch.nn.Linear(8,1)
        )
```

```
model = NeuralNetRegressor(2,8)

loss_function = torch.nn.MSELoss()

optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

for epoch in range(num_epochs):
    y_pred = model(x)
    loss = loss_function(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Retrieve all the weights in the model

Zero the gradients
Backpropagation
Update the weights

## Convolutional Neural Networks

An extra convolutional layer is used to reduce features. Could have more than one output channels which can be concatenated to form the output. **Stride** - how many squares the kernel moves per step
**Padding** - extra layer of 0s around the layer

$$\text{Output} = \left\lfloor \frac{\text{Input size} - \text{Kernel} + 2 \times \text{Padding}}{\text{Stride}} \right\rfloor + 1 \quad (1)$$

**Pooling Layer** - downsamples feature maps, eg. Max-Pool, Average-Pool, Sum-Pool

## Recurrent Neural Networks

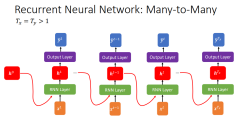
Process sequential data using **one-hot encoding** with a **hidden state** ($h^t$) to obtain contextual information. At time step $t$,

$$h^t = g^{[h]}((W^{[xh]})^T x^t + (W^{[hh]})^T h^{t-1})$$

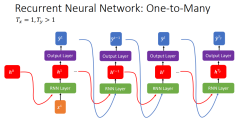
$$\hat{y}^t = g^{[y]}((W^{[hy]})^T h^t)$$

## Types

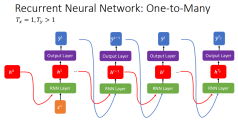
1. Many-to-many (equal): $T_x = T_y > 1$ (same number of inputs as outputs)




2. Many-to-One: $T_x > 1, T_y = 1$




3. One-to-Many: $T_x = 1, T_y > 1$ (input is output of previous layer)




4. Many-to-many (unequal): $T_x \neq T_y, T_x > 1, T_y > 1$ (<BEGIN> token to start output)

**Properties:** Captures context info from previous time steps, but output at time step $t$ must wait for all steps before $t$ to complete - not parallelism friendly.

## Self-Attention

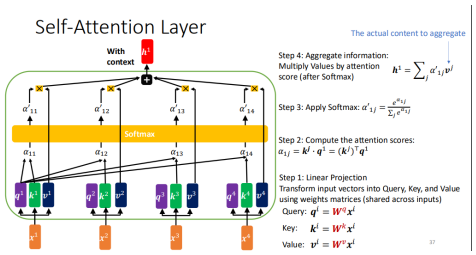
**Attention Score** ($\alpha$) - determine how much focus each part of the input sequence ($x_j$) should receive when processing a specific element ($x_i$).

$$\alpha_{ij} = k^j \cdot q^i = (k^j)^T q^i$$

Query: $q^i = W^q x^i$. Key: $k^j = W^k x^j$. $W^q$ and $W^k$ are both trainable weight matrices that enables the model to extract useful features for attention score calculation from the inputs. Can be done in parallel.

## Self-Attention Layer

1. Linear Projection - transform input vectors into Query, Key, and Value using matrices
   (a) $Q = W^q X,\ K = W^k X,\ V = W^v X$
2. Compute attention scores
   (a) $A = K^T Q$
3. Apply Softmax ($A' = softmax(A)$)
4. Multiply Values by post-Softmax attention score ($H = VA'$)




## Positional Encoding

Explicitly injects positional information into original input features.

$$x'^i = x^i + PE^i$$

Using sin and cos functions: $PE(i, 2k) = \sin\left(\frac{1}{10000^{\frac{2k}{d}}}\right)$

$$PE(i, 2k+1) = \cos\left(\frac{1}{10000^{\frac{2k}{d}}}\right)$$

## Issues with Deep Learning

- Overfitting
  - Dropout: Randomly set neurons output to 0
  - Early Stopping: Stop when validation loss starts increasing
- Vanishing Gradient
  - Change Activation Functions
- Exploding Gradient
  - Gradient Clipping

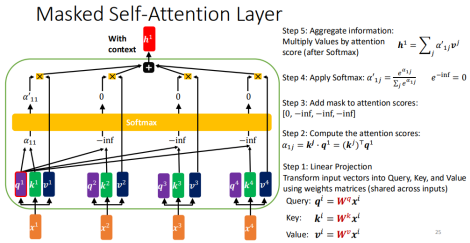
## Attention Neural Networks

**Types:**

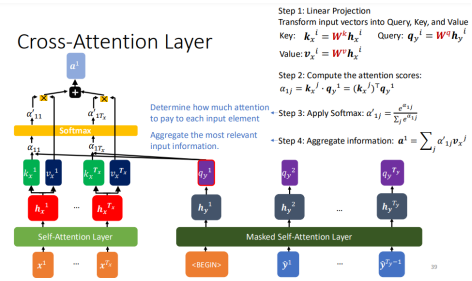
- Many-to-Many ($T_x = T_y > 1$)

- Many-to-One ($T_x > 1, T_y = 1$): Add learnable input vector <SUM> and use $h^{SUM}$ to predict output

- One-to-Many ($T_x = 1, T_y > 1$): $x^1, \hat{y}^1, \hat{y}^2, \ldots \hat{y}^{t-1}$ are used to generate $h^t$.
  - **Masked Self-Attention Layer**: Compute hidden state using only one input value
  - Add mask to attention score - let desired value be 0, the rest be $-\infty$.




- Many-to-Many ($T_x \neq T_y, T_x > 1, T_y > 1$)
  - **Cross-Attention Layer**: Input information used by the decoder during prediction
  - Transform input vectors into key and value. Query is computed from the decoder's hidden state. Attention score is computed from key of input and query of decoder. Then, we determine how much attention to pay to each input element.




## Transformer

Deep neural net based on attention mechanism. Feed Forward Network - Linear Layer + ReLU + Linear Layer
Encoder Block - Self-Attention Layer + Feed Forward Network
Decoder Block - Masked Self Attention + Cross Attention + FF Network